

---

Supplementary Material for:  
Cross-Family Symbolic Verification for  
Contamination-Robust Selective Prediction on LLM Math Reasoning

---

Anonymous Authors<sup>1</sup>

---

<sup>1</sup>Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Note. This document is the supplementary appendix accompanying the 4-page main paper. Section, table, figure, and listing numbering restart here.

The appendix supplements the 4-page main body with hyper-reproducible implementation details, full statistical methodology, the pre-registration text, additional results that did not fit in the main body, representative verifier case studies, and a broader-impact discussion.

## A. Extended Methods

### A.1. Solver and extractor configuration

The base solver is Qwen2.5-7B-Instruct-Turbo accessed via Together AI; for the false-acceptance analysis of self-verification the solver is the contamination-flagged Qwen2.5-Math-7B-Instruct (?). Solver sampling:  $T=0$  greedy for the plurality candidate;  $T=0.7$ , top- $p$  0.95,  $k=10$  for entropy and self-consistency baselines; max\_tokens=2048. The chain-of-thought prompt terminates with “The final answer is  $\boxed{\$}$ ”.

Cross-family extractors: Llama-3.3-70B-Instruct-Turbo, DeepSeek-V3, gpt-oss-120b, claude-sonnet-4-6, Qwen3-Coder-480B-A35B-Instruct, gpt-5-mini. Extractor sampling:  $T=0.2$ , max\_tokens=2048, system prompt fixing the output format to a single Python code block. The frozen extractor prompt is in ?? 1.

Listing 1. Frozen extractor prompt.

You are a mathematics verification engineer. You will be shown a math problem.  
Write a Python function

```
def verify(answer) -> bool:
```

that returns True if ‘answer’ (int, float, or sympy expression) satisfies the problem’s constraints, and False otherwise.

Hard rules:

1. Do NOT solve the problem. Do NOT write ‘answer == <literal>’.
2. Re-derive the problem’s constraints from the statement and check that ‘answer’ satisfies them.
3. You may import sympy, itertools, math, fractions.Fraction, and collections.Counter are pre-imported.
4. Wall-clock time < 10 seconds.
5. If the problem requires a proof or has no bounded answer check, return EXACTLY the string ‘UNVERIFIABLE’ and nothing else.
6. Total script under 60 lines.

Problem: <<<PROBLEM>>>

### A.2. Sandbox and probe set

The emitted script is executed in a Python subprocess sandbox with a SIGALRM wall-clock timeout of 10 seconds. Exception trapping maps any Exception or SystemExit to a False return. The candidate  $\hat{a}$  is cast through sympify before invocation; on SympifyError the candidate is passed as a string. Network sockets are not available in the subprocess.

The adversarial probe set  $\mathcal{P}(\hat{a}) = \{\hat{a} - 1, \hat{a} + 1, \hat{a} + 7, \hat{a} \times 2\}$  was selected on a 50-problem held-out development split of MATH-500 (strata-balanced, disjoint from any test set) and locked in the pre-registration. The four perturbations target trivial-True verifiers, additive-bias verifiers, modular-residue verifiers (the value 7 is coprime with  $\{2, 3, 4, 5, 6\}$ ), and multiplicative-scaling verifiers. For non-integer  $\hat{a}$  the deterministic fallback set  $\{0, 1, -1, 42, 100\} \setminus \{\hat{a}\}$  is used. A gold-relative variant of the same probe set is used only inside the benchmarking pipeline to compute the diagnostic Adv-FP column; the deployment-time filter never accesses the gold answer.

### A.3. PRM baselines

Qwen2.5-Math-PRM-7B and Skywork-o1-Open-PRM-Qwen-2.5-7B were run on Modal A100. For each problem and each of 10 cached solver samples, per-step rewards were aggregated using min-step, mean-step, product-step, and last-step aggregations as documented on each model card; problem-level score is the max across the 10 samples. PRMs were thresholded to match the X-SGRV top-tier coverage on each benchmark exactly.

Skywork-PRM requires `torch_dtype=torch.bfloat16`; on FP32 the model silently exhausts the A100’s 40 GB of GPU memory for sequences longer than  $\sim 1,500$  tokens. The PRM scoring protocols were validated against published ProcessBench numbers on a 48-problem ProcessBench subset, reproducing the published values within 5 macro-F1 points on both PRMs.

#### A.4. Verifier classification

A verifier is working if it (a) accepts the gold answer  $a$ , (b) rejects each of the four adversarial wrong candidates  $\{a - 1, a + 1, a + 7, 42\}$ , (c) halts within 10s, and (d) raises no unhandled exception. A verifier that accepts  $a$  but also accepts some adversarial candidate is trivial-or-broken. A verifier that fails to accept  $a$  is wrong-spec.

## B. Statistical Methodology and Pre-registration

Confidence intervals. 95% Clopper-Pearson exact binomial intervals via SciPy 1.13’s `binomtest().proportion_ci(method="exact")`. Bootstrap AUROC intervals use 1,000 resamples drawn with replacement.

Hypothesis tests. Matched-binary comparisons use McNemar’s exact mid- $p$  test via `statsmodels.stats.contingency_tables.mcnemar(exact=True)` (?). The  $2 \times 2$  ablation uses Fisher’s exact test on the contingency table (44, 0; 276, 334), yielding  $p < 10^{-8}$ .

Pre-registration text (committed before the  $n=498$  scale-up):

Hypothesis: strict two-way consensus (Llama  $\cap$  DeepSeek-V3) achieves top-tier precision  $\geq 99\%$  on the full MATH-500. Branches: (A) precision  $\geq 99\%$ , claim publishable; (B)  $95\% \leq \text{precision} < 99\%$ , report exact number, no overclaim; (C) precision  $< 95\%$ , audit each false acceptance, report ablations; halt-and-audit if precision  $< 90\%$ . Locked parameters: adversarial probe set  $\{\hat{a} \pm 1, \hat{a} + 7, \hat{a} \times 2\}$  (integer fallback  $\{0, 1, -1, 42, 100\} \setminus \{\hat{a}\}$ ); 10s sandbox timeout; working-verifier definition per Section A.4.

The realized result was  $171/171 = 1.000$ ,  $[0.979, 1.000]$ , fulfilling Branch A.

## C. Extended Results

### C.1. Same-model FAR by difficulty (Table C.1)

Table C.1. Same-model SymPy false acceptance rate on MATH-500 by difficulty. Solver and verifier are both Qwen2.5-Math-7B-Instruct.  $n$  = number of AllPass verdicts at each level. Total AllPass:  $n = 320$ .

| Level | L1   | L2   | L3    | L4    | L5    | Overall |
|-------|------|------|-------|-------|-------|---------|
| FAR   | 1.4% | 7.7% | 13.1% | 17.2% | 30.8% | 13.8%   |
| $n$   | 71   | 65   | 61    | 58    | 65    | 320     |

### C.2. $2 \times 2$ ablation: independence is the operative axis

A  $2 \times 2$  design crossed independence (solution- vs. specification-grounded extraction) with randomization (random evaluation points vs. deterministic symbolic checks). Both spec-grounded cells produced 0 false accepts (0/165, 0/169); both solution-grounded cells produced FAR  $\approx 13.8\%$ . Fisher’s exact on the contingency table (44, 0; 276, 334) yields  $p < 10^{-8}$  in favor of the independence axis.

### C.3. Agreement/entropy signal collapse on contamination-clean benchmarks (Table C.2)

On AIME 2025 the solver is uniformly uncertain ( $\geq 8$  distinct samples on nearly every problem), so the 4/4 self-consistency tier is empty. On CleanMath the solver is uniformly and confidently wrong (12% accuracy), and the NLI variant of semantic entropy collapses paraphrases of the same wrong answer into a single confident-looking cluster (5/59 correct).

Table C.2. Selective-prediction signals on contamination-assisted MATH-500 vs. contamination-clean AIME 2025 and CleanMath. “—” = empty top tier. Solver: Qwen2.5-7B-Instruct-Turbo.

| Method                 | MATH-500 | AIME 2025 | CleanMath |
|------------------------|----------|-----------|-----------|
| Verbalized             | 131/166  | 3/16      | 4/12      |
| Self-Consistency 4/4   | 105/118  | —         | —         |
| Template SGRV          | 59/59    | —         | —         |
| ExeVer same-model      | 276/320  | —         | —         |
| SE SymPy (zero-ent)    | 101/112  | —         | 1/6       |
| SE NLI (zero-ent)      | 114/131  | 1/16      | 5/59      |
| $p(\text{True})$ AUROC | 0.562    | 0.463     | 0.474     |

Table C.3. Per-extractor verifier classes on MATH-175. “Trivial-or-broken” equals the headline rate at which the deployment-time filter intervenes. Total  $n=175$  per row.

| Extractor         | Working | Wrong-spec | Trivial-or-broken | Unverif. |
|-------------------|---------|------------|-------------------|----------|
| Llama-3.3-70B     | 119     | 21         | 13                | 22       |
| DeepSeek-V3       | 124     | 18         | 11                | 22       |
| Claude Sonnet 4.6 | 112     | 23         | 18                | 22       |
| gpt-oss-120b      | 98      | 30         | 24                | 23       |
| Qwen3-Coder-480B  | 105     | 27         | 19                | 24       |
| GPT-5-mini        | 102     | 29         | 21                | 23       |

C.4. Per-extractor verifier classification on MATH-175 (Table C.3)

C.5. Lab-agnostic five-way consensus

Adding extractors monotonically shrinks the accepted tier on stratified MATH-175 while preserving precision: 73 (Llama)  $\rightarrow$  72 (Llama  $\cap$  DeepSeek)  $\rightarrow$  69 (3-way)  $\rightarrow$  69 (4-way)  $\rightarrow$  69 (5-way)  $\rightarrow$  67 (6-way with GPT-5-mini), all at precision 1.000.

C.6. Putnam scope test

On a 100-problem random sample of PutnamBench, the proof-mode prompt permits explicit UNVERIFIABLE refusals. Refusal rates: Claude Sonnet 4.6 51/100, GPT-5-mini 35/100, DeepSeek-V3 44/100, Llama-3.3-70B 39/100. Among non-refused problems, working-verifier-and-gold-accepted: Claude 19/49, GPT-5-mini 8/65, DeepSeek 14/56, Llama 11/61.

C.7. Lean compile-rate audit

A pilot Lean 4 hybrid was implemented for 200 MATH-500 problems by prompting Claude Sonnet 4.6 to emit, alongside the SymPy verifier, a Lean 4 theorem statement of the form theorem problem : <constraint> with placeholder by sorry. Compile rate against Mathlib4 4.10.0 was 158/200 = 78.9%; failures were predominantly import resolution and unicode-symbol parsing, not type-mismatch errors. This rate motivates the hybrid SymPy-plus-Lean follow-up; it is not used in any X-SGRV result.

## D. Verifier Case Studies

D.1. Working compound verifier (MATH-500, Level 4 algebra)

Problem (paraphrased): Find the product of all real roots of  $x^2 - 5x + 6 = 0$ .

```
def verify(answer):
    try:
        x = sympy.symbols('x')
        roots = sympy.solve(x**2 - 5*x + 6, x)
        product = sympy.prod(roots)
        return sympy.simplify(sympy.sympify(answer) - product) == 0
    except Exception:
        return False
```

## D.2. Working enumeration verifier (CleanMath/HMMT-Feb 2025)

```

def verify(answer):
    try:
        n = int(str(answer).strip())
        count = 0
        for a in range(1, 10):
            for b in range(0, 10):
                for c in range(0, 10):
                    if (a + b + c) == 12 and a*b*c % 7 == 0:
                        count += 1
        return n == count
    except Exception:
        return False

```

## D.3. Broken verifier rejected by the deployment-time filter

The following script accepts the gold answer but also accepts  $\hat{a} + 7$  and is rejected by the deployment-time filter:

```

def verify(answer):
    try:
        return int(str(answer).strip()) % 7 == 0
    except Exception:
        return False

```

## E. Limitations and Broader Impact

**Operational scope.** X-SGRV is a structurally-grounded selective-prediction signal, not a continuous-score uncertainty estimator. Top-tier coverage is single-digit on contamination-clean competition mathematics by design, because structural verifiability of a competition problem is a property of the problem, not of solver confidence. On Omni-MATH hard-100 (difficulty 9.0–9.5) zero working verifiers are produced, defining the operational ceiling on non-proof problems. On PutnamBench-style proof problems the system self-detects scope by emitting UNVERIFIABLE on roughly half of a 100-problem sample.

**Reliance on a strong cross-family extractor.** The empirical results require a  $\geq 70\text{B}$  cross-family extractor; at the 7B scale the working-verifier rate on MATH-500 is below 15%. The asymmetric extractor/solver pairing is consistent with centralized-governance safety patterns (??) but is a deployment cost.

**Independence assumption.** The consensus argument relies on an independence assumption between extractor families. The empirical 0/171 at the  $n=498$  scale places a lower bound on effective independence but is not a structural proof. The proposed falsification protocol — empirical pairwise correlation between extractor verifier outputs vs. the independence-bound prediction — is described as future work in the discussion.

**Broader impact.** X-SGRV supports abstention on uncertain mathematical reasoning, not substitute for human review. Deploying it in high-stakes settings (automated grading, safety-critical scientific computation, medical decision support) without an explicit human-in-the-loop fallback would invert the safety pattern the system is designed to support: the right deployment posture is to escalate, not commit, when the verifier abstains. Top-tier acceptances on contamination-clean benchmarks (e.g., 4/4 on CleanMath) are correct in our experiments, but a deployment policy that treats individual accepts as authoritative without human review would be misuse.

**Energy and compute footprint.** The full  $n=498$  MATH-500 scale-up consumed approximately  $3.1 \times 10^6$  output tokens across all extractor and solver calls, equivalent to roughly \$45 in API costs at 2026-04 list prices. PRM baselines required approximately 4 A100-hours on Modal. No model fine-tuning was performed.